

```

import os
import sqlite3
import logging
from datetime import datetime, timedelta
from io import BytesIO
import threading
import time

from PIL import Image, ImageDraw, ImageFont
import telebot
from telebot import types
from flask import Flask, render_template_string, jsonify, request

# ===== SOZLAMALAR =====

BOT_TOKEN = os.environ.get("BOT_TOKEN", "YOUR_BOT_TOKEN_HERE")
WEB_SECRET = os.environ.get("WEB_SECRET", "secret123")
WEB_PORT = int(os.environ.get("WEB_PORT", 5000))

bot = telebot.TeleBot(BOT_TOKEN)
app = Flask(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(message)s')

# ===== DATABASE =====

DB_PATH = 'cafe_debts.db'

def init_db():
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    c = conn.cursor()

    c.execute('''CREATE TABLE IF NOT EXISTS users (
        user_id INTEGER PRIMARY KEY,
        full_name TEXT,
        username TEXT,
        role TEXT DEFAULT 'worker',
        added_at TEXT,
        added_by INTEGER
    )''')

    c.execute('''CREATE TABLE IF NOT EXISTS products (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        supplier_name TEXT,

```

```

        total_price REAL NOT NULL,
        paid_amount REAL DEFAULT 0,
        due_date TEXT,
        photo_file_id TEXT,
        note TEXT,
        -- Hisob raqamlar
        naqd_account TEXT,
        online_account TEXT,
        online_bank TEXT,
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL,
        created_by INTEGER
    )'''

c.execute('''CREATE TABLE IF NOT EXISTS payments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_id INTEGER NOT NULL,
    amount REAL NOT NULL,
    payment_type TEXT DEFAULT 'cash',
    receipt_file_id TEXT,
    note TEXT,
    paid_at TEXT NOT NULL,
    added_by INTEGER,
    FOREIGN KEY (product_id) REFERENCES products(id)
)''')

c.execute('''CREATE TABLE IF NOT EXISTS reminders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_id INTEGER NOT NULL,
    remind_at TEXT NOT NULL,
    sent INTEGER DEFAULT 0,
    FOREIGN KEY (product_id) REFERENCES products(id)
)''')

conn.commit()
conn.close()

def get_db():
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    return conn

# ===== ADMIN BOSHQARUVI =====

def get_admins():
    db = get_db()
    rows = db.execute("SELECT user_id FROM users WHERE role='admin'").fetchall()

```

```

    db.close()
    return [r['user_id'] for r in rows]

def is_admin(user_id):
    db = get_db()
    row = db.execute("SELECT role FROM users WHERE user_id=?", (user_id,)).fetcho
    db.close()
    return row and row['role'] == 'admin'

def is_allowed(user_id):
    db = get_db()
    row = db.execute("SELECT user_id FROM users WHERE user_id=?", (user_id,)).fet
    db.close()
    return row is not None

def register_admin(user_id, full_name, username):
    db = get_db()
    existing = db.execute("SELECT user_id FROM users WHERE user_id=?", (user_id,))
    if not existing:
        db.execute("INSERT INTO users (user_id, full_name, username, role, added_
                    (user_id, full_name, username or '', datetime.now().isoformat(
                    db.commit()
    db.close()

# ===== USER STATE =====

user_states = {}

def set_state(uid, state, data=None):
    user_states[uid] = {'state': state, 'data': data or {}}

def get_state(uid):
    return user_states.get(uid, {'state': None, 'data': {}})

def clear_state(uid):
    user_states.pop(uid, None)

# ===== CHEK RASMI GENERATOR =====

def generate_receipt(product, payments, remaining):
    W = 600
    pay_count = len(payments)
    H = 860 + pay_count * 62

    img = Image.new('RGB', (W, H), '#0d1117')
    draw = ImageDraw.Draw(img)

```

```

for i in range(H):
    r = int(13 + (20 - 13) * i / H)
    g = int(17 + (28 - 17) * i / H)
    b = int(23 + (45 - 23) * i / H)
    draw.line([(0, i), (W, i)], fill=(r, g, b))

def rr(x1, y1, x2, y2, r=12, fill=None, outline=None, w=2):
    draw.rounded_rectangle([x1, y1, x2, y2], radius=r, fill=fill, outline=out

def load_font(size, bold=False):
    paths = [
        f"/usr/share/fonts/truetype/dejavu/DejaVuSans{'Bold' if bold else ''}
        f"/usr/share/fonts/truetype/liberation/LiberationSans-{'Bold' if bold
    ]
    for p in paths:
        try:
            return ImageFont.truetype(p, size)
        except:
            continue
    return ImageFont.load_default()

f36b = load_font(36, True)
f24b = load_font(24, True)
f20 = load_font(20)
f20b = load_font(20, True)
f16 = load_font(16)
f14 = load_font(14)

# HEADER
rr(15, 15, W-15, 115, r=20, fill='#161b22', outline='#30363d')
draw.text((W//2, 50), "☕ KAFE NASIYA DAFTARI", font=f24b, fill='#f0883e', an
draw.text((W//2, 88), "Tovar Hisobi & To'lov Cheki", font=f16, fill='#8b949e'

for i in range(0, W-30, 8):
    color = '#f0883e' if (i // 8) % 3 == 0 else '#21262d'
    draw.rectangle([i+15, 123, i+21, 126], fill=color)

# TOVAR MA'LUMOTI
y = 140
rr(15, y, W-15, y+125, r=15, fill='#161b22', outline='#30363d')
draw.text((35, y+12), "📦 TOVAR", font=f14, fill='#8b949e')
draw.text((35, y+38), product['name'], font=f24b, fill='#e6edf3')
if product.get('supplier_name'):
    draw.text((35, y+75), f"🏠 {product['supplier_name']}", font=f16, fill='
if product.get('due_date'):
    draw.text((35, y+98), f"📅 Muddat: {str(product['due_date'])[:10]}", fon
draw.text((W-35, y+38), f"{product['total_price']:.0f}", font=f24b, fill='#f

```

```

draw.text((W-35, y+68), "so'm (jami)", font=f14, fill='#8b949e', anchor='ra')

# HISOB RAQAMLAR
y += 138
if product.get('naqd_account') or product.get('online_account'):
    rr(15, y, W-15, y+90, r=15, fill='#161b22', outline='#21262d')
    draw.text((35, y+12), "🏠 HISOB RAQAMLAR", font=f14, fill='#8b949e')
    ya = y + 35
    if product.get('naqd_account'):
        draw.text((35, ya), f"💰 Naqd: {product['naqd_account']}", font=f16,
            ya += 25
    if product.get('online_account'):
        bank = product.get('online_bank', 'Online')
        draw.text((35, ya), f"🏦 {bank}: {product['online_account']}", font=f
y += 103

# TO'LOVLAR TARIXI
draw.text((35, y+5), "📅 TO'LOVLAR TARIXI", font=f16, fill='#8b949e')
y += 30
box_h = pay_count * 62 + 20 if pay_count > 0 else 55
rr(15, y, W-15, y+box_h, r=15, fill='#161b22', outline='#30363d')
y += 12

if not payments:
    draw.text((W//2, y+18), "Hali to'lov kiritilmagan", font=f16, fill='#484f
y += 40
else:
    for i, p in enumerate(payments):
        row_color = '#1c2128' if i % 2 == 0 else '#161b22'
        rr(25, y, W-25, y+54, r=8, fill=row_color)
        ptype_icon = "🏦" if p['payment_type'] == 'click' else "💰"
        ptype_label = "Online" if p['payment_type'] == 'click' else "Naqd"
        dt = str(p['paid_at'][:16]).replace('T', ' ')
        draw.text((42, y+8), f"{ptype_icon} {dt}", font=f14, fill='#8b949e')
        draw.text((42, y+30), ptype_label, font=f14,
            fill='#3fb950' if p['payment_type'] == 'cash' else '#58a6ff
        draw.text((W-40, y+20), f"+{p['amount']:,.0f} so'm", font=f20b, fill=
y += 58

y += 20

# STATISTIKA
paid = product['paid_amount']
total = product['total_price']
percent = min((paid / total * 100) if total > 0 else 0, 100)

rr(15, y, W-15, y+185, r=15, fill='#161b22', outline='#30363d')

```

```

draw.text((35, y+15), "🇺🇦 HOLAT", font=f14, fill='#8b949e')

bx, by = 35, y+45
bw = W - 70
bh = 22
rr(bx, by, bx+bw, by+bh, r=11, fill='#21262d')
fw = int(bw * percent / 100)
if fw > 12:
    bar_color = '#3fb950' if percent >= 90 else '#f0883e' if percent >= 50 el
    rr(bx, by, bx+fw, by+bh, r=11, fill=bar_color)

draw.text((W//2, by+bh+18), f"{percent:.1f}% to'langan", font=f16, fill='#8b
draw.text((35, y+108), "✅ To'langan:", font=f20, fill='#8b949e')
draw.text((W-35, y+108), f"{paid:,.0f} so'm", font=f20b, fill='#3fb950', anch
draw.text((35, y+142), "🕒 Qolgan qarz:", font=f20, fill='#8b949e')
if remaining <= 0:
    draw.text((W-35, y+142), "✅ To'liq to'landi!", font=f20b, fill='#3fb950
else:
    draw.text((W-35, y+142), f"{remaining:,.0f} so'm", font=f24b, fill='#f851

y += 195

draw.line([(35, y), (W-35, y)], fill='#30363d', width=1)
now = datetime.now().strftime('%d.%m.%Y %H:%M')
draw.text((W//2, y+18), f"🕒 {now}", font=f14, fill='#484f58', anchor='mm')
draw.text((W//2, y+40), "☕ Kafe Nasiya Daftari Bot", font=f14, fill='#30363d

rr(2, 2, W-3, H-3, r=22, outline='#f0883e', w=2)

buf = BytesIO()
img.save(buf, format='PNG')
buf.seek(0)
return buf

# ===== KLAVIATURA =====

def admin_menu():
    m = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    m.add(
        types.KeyboardButton("✚ Yangi tovar"),
        types.KeyboardButton("📦 Tovarlar"),
        types.KeyboardButton("📄 To'lov kiritish"),
        types.KeyboardButton("🇺🇦 Umumiy holat"),
        types.KeyboardButton("👥 Xodimlar"),
        types.KeyboardButton("🌐 Web sahifa")
    )
    return m

```

```

def worker_menu():
    m = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    m.add(
        types.KeyboardButton("📦 Tovarlar"),
        types.KeyboardButton("📊 Umumiy holat")
    )
    return m

def get_menu(uid):
    return admin_menu() if is_admin(uid) else worker_menu()

def cancel_kb():
    m = types.ReplyKeyboardMarkup(resize_keyboard=True)
    m.add(types.KeyboardButton("❌ Bekor qilish"))
    return m

def skip_kb():
    m = types.ReplyKeyboardMarkup(resize_keyboard=True, row_width=2)
    m.add(types.KeyboardButton("⏩ O'tkazib yuborish"), types.KeyboardButton("❌"))
    return m

def products_markup(uid, action="view"):
    db = get_db()
    rows = db.execute("SELECT id, name, total_price, paid_amount FROM products OR")
    db.close()
    if not rows:
        return None, []
    m = types.InlineKeyboardMarkup(row_width=1)
    for r in rows:
        rem = r['total_price'] - r['paid_amount']
        icon = "✅" if rem <= 0 else "🔴"
        m.add(types.InlineKeyboardButton(
            f"{icon} {r['name']} | {rem:,.0f} so'm qolgan",
            callback_data=f"{action}:{r['id']}"
        ))
    return m, rows

# ===== ESLATMA TIZIMI =====

def reminder_loop():
    while True:
        try:
            db = get_db()
            now = datetime.now().isoformat()
            reminders = db.execute(
                "SELECT r.id, r.product_id, p.name, p.total_price, p.paid_amount

```

```

        "FROM reminders r JOIN products p ON r.product_id=p.id "
        "WHERE r.sent=0 AND r.remind_at <= ?", (now,)
    ).fetchall()

    for rem in reminders:
        remaining = rem['total_price'] - rem['paid_amount']
        if remaining > 0:
            admins = get_admins()
            text = (
                f"🕒 *ESLATMA!*\\n\\n"
                f"📦 *{rem['name']}* uchun to'lov muddati yaqinlashdi!\\n\\n"
                f"$ Qolgan qarz: *{remaining:,.0f}* so'm*\\n\\n"
                f"🕒 {datetime.now().strftime('%d.%m.%Y %H:%M')}\\n\\n"
            )
            for admin_id in admins:
                try:
                    bot.send_message(admin_id, text, parse_mode='Markdown')
                except:
                    pass

            db.execute("UPDATE reminders SET sent=1 WHERE id=?", (rem['id'],))
            db.commit()
        db.close()
    except Exception as e:
        logging.error(f"Reminder error: {e}")
    time.sleep(60)

# ===== /start =====

@bot.message_handler(commands=['start'])
def cmd_start(msg):
    uid = msg.from_user.id
    name = msg.from_user.first_name
    uname = msg.from_user.username or ''

    admins = get_admins()

    if not admins:
        register_admin(uid, name, uname)
        bot.send_message(uid,
            f"👤 Assalomu alaykum, *{name}*!\\n\\n"
            f"Siz *Admin* sifatida ro'yxatdan o'tdingiz.\\n\\n"
            f"☕ *Kafe Nasiya Daftari* — barcha nasiya va to'lovlaringiz shu yerdan"
            f"📄 parse_mode='Markdown', reply_markup=admin_menu())
    elif not is_allowed(uid):
        bot.send_message(uid,
            f"🔒 Siz tizimda yo'qsiz.\\n\\nAdmin sizni qo'shishi kerak.")

```

```

else:
    role = "👑 Admin" if is_admin(uid) else "👤 Xodim"
    bot.send_message(uid,
        f"👋 Xush kelibsiz, *{name}*\nRolingiz: {role}",
        parse_mode='Markdown', reply_markup=get_menu(uid))

# ===== TOVAR QO'SHISH =====

@bot.message_handler(func=lambda m: m.text == "➕ Yangi tovar")
def add_product(msg):
    uid = msg.from_user.id
    if not is_admin(uid):
        return
    set_state(uid, 'prod_name')
    bot.send_message(uid, "📦 *Tovar nomini kiriting:*\n\n_Misol: Tovuq go'shti_"
        parse_mode='Markdown', reply_markup=cancel_kb())

@bot.message_handler(func=lambda m: m.text == "📦 Tovarlar")
def show_products(msg):
    uid = msg.from_user.id
    if not is_allowed(uid):
        return
    markup, rows = products_markup(uid, "view")
    if not rows:
        bot.send_message(uid, "🇷🇺 Hali hech qanday tovar yo'q.", reply_markup=get
        return
    bot.send_message(uid, "📦 *Tovarlar ro'yxati:*\n\nTovar ustiga bosing → bataf
        parse_mode='Markdown', reply_markup=markup)

@bot.message_handler(func=lambda m: m.text == "📦 To'lov kiritish")
def pay_start(msg):
    uid = msg.from_user.id
    if not is_admin(uid):
        return
    markup, rows = products_markup(uid, "pay")
    if not rows:
        bot.send_message(uid, "🇷🇺 Avval tovar qo'shing.", reply_markup=admin_menu
        return
    bot.send_message(uid, "📦 *Qaysi tovar uchun to'lov?*",
        parse_mode='Markdown', reply_markup=markup)

@bot.message_handler(func=lambda m: m.text == "🇮🇹 Umumiy holat")
def total_stats(msg):
    uid = msg.from_user.id
    if not is_allowed(uid):
        return
    db = get_db()

```

```

row = db.execute("SELECT COUNT(*), SUM(total_price), SUM(paid_amount) FROM pr
db.close()
if not row or not row[0]:
    bot.send_message(uid, "🇺🇦 Ma'lumot yo'q.", reply_markup=get_menu(uid))
    return
count = row[0] or 0
total = row[1] or 0
paid = row[2] or 0
remaining = total - paid
pct = (paid / total * 100) if total > 0 else 0
bar = "█" * int(pct / 5) + "░" * (20 - int(pct / 5))
bot.send_message(uid,
    f"🇺🇦 *Umumiy holat*\n\n"
    f"📦 Tovarlar: *{count} ta*\n\n"
    f"$ Jami nasiya: *{total:,.0f} so'm*\n"
    f"✅ To'langan: *{paid:,.0f} so'm*\n"
    f"🔴 Qolgan qarz: *{remaining:,.0f} so'm*\n\n"
    f"`{bar}`\n"
    f"*{pct:.1f}% to'langan*",
    parse_mode='Markdown', reply_markup=get_menu(uid))

@bot.message_handler(func=lambda m: m.text == "🌐 Web sahifa")
def web_page_info(msg):
    uid = msg.from_user.id
    if not is_admin(uid):
        return
    host = os.environ.get("WEB_HOST", "http://localhost:5000")
    bot.send_message(uid,
        f"🌐 *Web Dashboard*\n\n"
        f"🔗 Havola: `{host}`\n\n"
        f"🔑 Parol: `{WEB_SECRET}`\n\n"
        f"_Bu havoladan barcha tovarlarni, to'lovlarni va statistikani ko'rishing"
        parse_mode='Markdown', reply_markup=admin_menu())

@bot.message_handler(func=lambda m: m.text == "👥 Xodimlar")
def workers_menu(msg):
    uid = msg.from_user.id
    if not is_admin(uid):
        return
    db = get_db()
    workers = db.execute("SELECT user_id, full_name, username, role FROM users").
    db.close()

    text = "👥 *Foydalanuvchilar:*\n\n"
    for w in workers:
        role_icon = "👑" if w['role'] == 'admin' else "👤"
        uname = f"@{w['username']}" if w['username'] else "-"

```

```

text += f"{role_icon} *{w['full_name']}* ({uname})\n"

markup = types.InlineKeyboardMarkup()
markup.add(types.InlineKeyboardButton("✚ Xodim qo'shish", callback_data="add"))
markup.add(types.InlineKeyboardButton("🗑 Xodim o'chirish", callback_data="re"))
bot.send_message(uid, text, parse_mode='Markdown', reply_markup=markup)

@bot.message_handler(func=lambda m: m.text == "❌ Bekor qilish")
def cancel_action(msg):
    uid = msg.from_user.id
    clear_state(uid)
    bot.send_message(uid, "❌ Bekor qilindi.", reply_markup=get_menu(uid))

@bot.message_handler(func=lambda m: m.text == "▶▶ O'tkazib yuborish")
def skip_step(msg):
    uid = msg.from_user.id
    st = get_state(uid)
    state = st['state']
    data = st['data']

    if state == 'prod_supplier':
        set_state(uid, 'prod_price', data)
        bot.send_message(uid, "💰 *Jami narxini kiriting (so'mda):*\n_Misol: 1500",
                          parse_mode='Markdown', reply_markup=skip_kb())
    elif state == 'prod_due':
        set_state(uid, 'prod_naqd_acc', data)
        bot.send_message(uid, "💵 *Naqd to'lov uchun hisob raqam/karta:*\n_(O'tka",
                          parse_mode='Markdown', reply_markup=skip_kb())
    elif state == 'prod_naqd_acc':
        set_state(uid, 'prod_online_bank', data)
        bot.send_message(uid, "🏦 *Online to'lov banki nomi:*\n_(O'tkazish mumkin",
                          parse_mode='Markdown', reply_markup=skip_kb())
    elif state == 'prod_online_bank':
        set_state(uid, 'prod_online_acc', data)
        bot.send_message(uid, "💳 *Online to'lov hisob raqami/karta:*\n_(O'tkazis",
                          parse_mode='Markdown', reply_markup=skip_kb())
    elif state == 'prod_online_acc':
        set_state(uid, 'prod_photo', data)
        bot.send_message(uid, "📷 *Tovar rasmini yuboring:*\n_(O'tkazish mumkin)",
                          parse_mode='Markdown', reply_markup=skip_kb())
    elif state == 'prod_photo':
        set_state(uid, 'prod_note', data)
        bot.send_message(uid, "📝 *Izoh yozing:*\n_(O'tkazish mumkin)",
                          parse_mode='Markdown', reply_markup=skip_kb())
    elif state == 'prod_note':
        _save_product(uid, data)

```

```

# TAHRIRLASH uchun skip
elif state == 'edit_supplier':
    _finish_edit(uid, data, 'supplier_name', None)
elif state == 'edit_due':
    _finish_edit(uid, data, 'due_date', None)
elif state == 'edit_naqd_acc':
    _finish_edit(uid, data, 'naqd_account', None)
elif state == 'edit_online_bank':
    _finish_edit(uid, data, 'online_bank', None)
elif state == 'edit_online_acc':
    _finish_edit(uid, data, 'online_account', None)
elif state == 'edit_photo':
    _ask_edit_note(uid, data)
elif state == 'edit_note':
    _finish_edit(uid, data, 'note', None)

# ===== MATN HANDLERLARI =====

@bot.message_handler(content_types=['text', 'photo'])
def handle_all(msg):
    uid = msg.from_user.id
    if not is_allowed(uid):
        bot.send_message(uid, "🔒 Kirish taqiqlangan. Admindan ruxsat so'rang.")
        return

    st = get_state(uid)
    state = st['state']
    data = st['data']

    # === TOVAR QO'SHISH ===
    if state == 'prod_name':
        name = msg.text.strip() if msg.text else ''
        if len(name) < 2:
            bot.send_message(uid, "⚠️ Iltimos, tovar nomini to'g'ri kiriting.")
            return
        data['name'] = name
        set_state(uid, 'prod_supplier', data)
        bot.send_message(uid, f"✅ Tovar: {name}\n📦 *Yetkazuvchi ismini kiriting*"
                             parse_mode='Markdown', reply_markup=skip_kb())

    elif state == 'prod_supplier':
        data['supplier'] = msg.text.strip() if msg.text else ''
        set_state(uid, 'prod_price', data)
        bot.send_message(uid, "💰 *Jami narxini kiriting (so'mda):*\n_Misol: 1500*"
                             parse_mode='Markdown', reply_markup=skip_kb())

    elif state == 'prod_price':

```

```

try:
    price = float((msg.text or '').replace(',', ' ').replace(' ', ''))
    if price <= 0:
        raise ValueError
except:
    bot.send_message(uid, "⚠️ To'g'ri raqam kiriting. _Misol: 15000000_",
        return
data['price'] = price
set_state(uid, 'prod_due', data)
bot.send_message(uid,
    "📅 *To'lov muddatini kiriting:*\n_Format: 25.12.2024_\n_(O'tkazish n
    parse_mode='Markdown', reply_markup=skip_kb())

elif state == 'prod_due':
    text = (msg.text or '').strip()
    try:
        due = datetime.strptime(text, '%d.%m.%Y').isoformat()
        data['due_date'] = due
    except:
        data['due_date'] = None
    set_state(uid, 'prod_naqd_acc', data)
    bot.send_message(uid,
        "💵 *Naqd to'lov uchun karta/hisob raqam:*\n_(O'tkazish mumkin)_\n\n
        parse_mode='Markdown', reply_markup=skip_kb())

elif state == 'prod_naqd_acc':
    data['naqd_account'] = msg.text.strip() if msg.text else None
    set_state(uid, 'prod_online_bank', data)
    bot.send_message(uid,
        "🏦 *Online to'lov banki:*\n_(O'tkazish mumkin)_\n\n_Misol: Payme, Cl
        parse_mode='Markdown', reply_markup=skip_kb())

elif state == 'prod_online_bank':
    data['online_bank'] = msg.text.strip() if msg.text else None
    set_state(uid, 'prod_online_acc', data)
    bot.send_message(uid,
        "💳 *Online to'lov hisob raqami:*\n_(O'tkazish mumkin)_",
        parse_mode='Markdown', reply_markup=skip_kb())

elif state == 'prod_online_acc':
    data['online_account'] = msg.text.strip() if msg.text else None
    set_state(uid, 'prod_photo', data)
    bot.send_message(uid, "📷 *Tovar rasmini yuboring:*\n_(O'tkazish mumkin)_
        parse_mode='Markdown', reply_markup=skip_kb())

elif state == 'prod_photo':
    if msg.photo:

```

```

        data['photo'] = msg.photo[-1].file_id
        set_state(uid, 'prod_note', data)
        bot.send_message(uid, "✎ *Izoh yozing:*\n_(O'tkazish mumkin)_",
                           parse_mode='Markdown', reply_markup=skip_kb())

elif state == 'prod_note':
    data['note'] = msg.text.strip() if msg.text else ''
    _save_product(uid, data)

# === TO'LOV ===
elif state == 'pay_amount':
    try:
        amount = float((msg.text or '').replace(',', '').replace(' ', ''))
        if amount <= 0:
            raise ValueError
    except:
        bot.send_message(uid, "⚠ To'g'ri summa kiriting.", reply_markup=canc)
        return

    db = get_db()
    prod = db.execute("SELECT total_price, paid_amount, naqd_account, online_
                        (data['product_id'],)).fetchone()
    db.close()

    if not prod:
        bot.send_message(uid, "❌ Tovar topilmadi.", reply_markup=admin_menu(
            clear_state(uid)
        return

    remaining = prod['total_price'] - prod['paid_amount']
    if amount > remaining:
        amount = remaining

    data['amount'] = amount
    data['naqd_account'] = prod['naqd_account']
    data['online_account'] = prod['online_account']
    data['online_bank'] = prod['online_bank']
    set_state(uid, 'pay_type', data)

    m = types.InlineKeyboardMarkup()
    m.add(
        types.InlineKeyboardButton("💵 Naqd pul", callback_data="ptype:cash")
        types.InlineKeyboardButton("🏠 Online/Karta", callback_data="ptype:cl
    )

    acc_info = ""
    if prod['naqd_account']:

```

```

        acc_info += f"\n💰 Naqd karta: `{prod['naqd_account']}`"
    if prod['online_account']:
        bank = prod['online_bank'] or 'Online'
        acc_info += f"\n🏠 {bank}: `{prod['online_account']}`"

    bot.send_message(uid,
        f"💰 Summa: *{amount:,.0f}* so'm*\n\n"
        f"🏠 *To'lov ma'lumotlari: *{acc_info if acc_info else ' Kiritilmagan'
        f"🏠 *To'lov turini tanlang: *",
        parse_mode='Markdown', reply_markup=m)

elif state == 'pay_receipt':
    if msg.photo:
        data['receipt'] = msg.photo[-1].file_id
        _save_payment(uid, data)

# === XODIM QO'SHISH ===
elif state == 'add_worker_id':
    try:
        worker_id = int((msg.text or '').strip())
    except:
        bot.send_message(uid, "⚠️ Telegram ID raqam bo'lishi kerak.", reply_m
        return
    set_state(uid, 'add_worker_name', {'worker_id': worker_id})
    bot.send_message(uid, "👤 *Xodim ismini kiriting: *", parse_mode='Markdown

elif state == 'add_worker_name':
    worker_name = (msg.text or '').strip()
    worker_id = data['worker_id']
    db = get_db()
    existing = db.execute("SELECT user_id FROM users WHERE user_id=?", (worke
    if existing:
        bot.send_message(uid, "⚠️ Bu foydalanuvchi allaqachon mavjud.", reply
    else:
        db.execute(
            "INSERT INTO users (user_id, full_name, username, role, added_at,
            (worker_id, worker_name, datetime.now().isoformat(), uid))
        db.commit()
        try:
            bot.send_message(worker_id,
                f"✅ Siz *Kafe Nasiya Daftari*ga xodim sifatida qo'shildingiz
                parse_mode='Markdown', reply_markup=worker_menu())
        except:
            pass
        bot.send_message(uid, f"✅ *{worker_name}* xodim sifatida qo'shildi!"
    db.close()
    clear_state(uid)

```

```

# === TAHRIRLASH ===
elif state == 'edit_name':
    new_val = msg.text.strip() if msg.text else ''
    if len(new_val) < 2:
        bot.send_message(uid, "⚠️ Nom juda qisqa.", reply_markup=cancel_kb())
        return
    _finish_edit(uid, data, 'name', new_val)

elif state == 'edit_supplier':
    _finish_edit(uid, data, 'supplier_name', msg.text.strip() if msg.text else '')

elif state == 'edit_price':
    try:
        price = float((msg.text or '').replace(',', '.').replace(' ', ''))
        if price <= 0:
            raise ValueError
    except:
        bot.send_message(uid, "⚠️ To'g'ri raqam kiriting.", reply_markup=cancel_kb())
        return
    _finish_edit(uid, data, 'total_price', price)

elif state == 'edit_due':
    text = (msg.text or '').strip()
    try:
        due = datetime.strptime(text, '%d.%m.%Y').isoformat()
    except:
        due = None
    _finish_edit(uid, data, 'due_date', due)

elif state == 'edit_naqd_acc':
    _finish_edit(uid, data, 'naqd_account', msg.text.strip() if msg.text else '')

elif state == 'edit_online_bank':
    _finish_edit(uid, data, 'online_bank', msg.text.strip() if msg.text else '')

elif state == 'edit_online_acc':
    _finish_edit(uid, data, 'online_account', msg.text.strip() if msg.text else '')

elif state == 'edit_photo':
    if msg.photo:
        data['new_photo'] = msg.photo[-1].file_id
        _ask_edit_note(uid, data)

elif state == 'edit_note':
    _finish_edit(uid, data, 'note', msg.text.strip() if msg.text else None)

```

```
# ===== TAHRIRLASH YORDAMCHI =====
```

```
def _ask_edit_note(uid, data):
    pid = data['product_id']
    if data.get('new_photo'):
        db = get_db()
        db.execute("UPDATE products SET photo_file_id=?, updated_at=? WHERE id=?"
                    (data['new_photo'], datetime.now().isoformat(), pid))
        db.commit()
        db.close()
    set_state(uid, 'edit_note', data)
    bot.send_message(uid, "📄 *Yangi izoh:* \n_(O'tkazish mumkin)_",
                      parse_mode='Markdown', reply_markup=skip_kb())
```

```
def _finish_edit(uid, data, field, value):
    pid = data['product_id']
    now = datetime.now().isoformat()
    db = get_db()

    if field == 'note' and value is None:
        value = ''

    db.execute(f"UPDATE products SET {field}=?, updated_at=? WHERE id=?", (value,
    db.commit()
    prod = db.execute("SELECT * FROM products WHERE id=?", (pid,)).fetchone()
    db.close()
    clear_state(uid)

    field_names = {
        'name': 'Nom', 'supplier_name': 'Yetkazuvchi', 'total_price': 'Narx',
        'due_date': 'Muddat', 'naqd_account': 'Naqd hisob', 'online_bank': 'Onlin
        'online_account': 'Online hisob', 'note': 'Izoh'
    }
    bot.send_message(uid,
        f"✅ *{field_names.get(field, field)}* yangilandi!\n\n"
        f"📦 *{prod['name']}*",
        parse_mode='Markdown', reply_markup=admin_menu())
```

```
# ===== SAQLASH =====
```

```
def _save_product(uid, data):
    now = datetime.now().isoformat()
    db = get_db()
    db.execute(
        "INSERT INTO products (name, supplier_name, total_price, paid_amount, due
        'naqd_account, online_account, online_bank, photo_file_id, note, created_
        "VALUES (?, ?, ?, 0, ?, ?, ?, ?, ?, ?, ?, ?)",
```

```

        (data.get('name'), data.get('supplier'), data.get('price'),
         data.get('due_date'), data.get('naqd_account'), data.get('online_account'),
         data.get('online_bank'), data.get('photo'), data.get('note'), now, now,
        )
    pid = db.execute("SELECT last_insert_rowid()").fetchone()[0]

    if data.get('due_date'):
        due = datetime.fromisoformat(data['due_date'])
        remind_at = (due - timedelta(days=1)).isoformat()
        db.execute("INSERT INTO reminders (product_id, remind_at) VALUES (?,?)",

    db.commit()
    db.close()
    clear_state(uid)

    acc_info = ""
    if data.get('naqd_account'):
        acc_info += f"\n💰 Naqd: `{data['naqd_account']}`"
    if data.get('online_account'):
        bank = data.get('online_bank', 'Online')
        acc_info += f"\n🏦 {bank}: `{data['online_account']}`"

    bot.send_message(uid,
        f"✅ *Tovar qo'shildi!* \n\n"
        f"📦 *{data.get('name')}* \n"
        f"💰 Jami: *{data.get('price', 0):,.0f}* so'm*"
        f"{acc_info} \n\n"
        f"To'lov kiritish uchun 📄 To'lov kiritish* tugmasini bosing.",
        parse_mode='Markdown', reply_markup=admin_menu())

def _save_payment(uid, data):
    product_id = data['product_id']
    amount = data['amount']
    ptype = data.get('ptype', 'cash')
    receipt = data.get('receipt')
    now = datetime.now().isoformat()

    db = get_db()
    db.execute("UPDATE products SET paid_amount = paid_amount + ?, updated_at=? W
                (amount, now, product_id))
    db.execute("INSERT INTO payments (product_id, amount, payment_type, receipt_f
                (product_id, amount, ptype, receipt, now, uid))
    db.commit()

    prod = db.execute("SELECT * FROM products WHERE id=?", (product_id,)).fetchon
    payments = db.execute("SELECT * FROM payments WHERE product_id=? ORDER BY pai
                (product_id,)).fetchall()

```

```

db.close()

remaining = prod['total_price'] - prod['paid_amount']
clear_state(uid)

bot.send_message(uid, "🧾 *Chek tayyorlanmoqda...*", parse_mode='Markdown')

try:
    prod_dict = dict(prod)
    pay_list = [dict(p) for p in payments]
    receipt_img = generate_receipt(prod_dict, pay_list, remaining)
    caption = (
        f"🧾 *{prod['name']}* - To'lov cheki\n\n"
        f"💰 To'langan: *{amount:,.0f}* so'm*\n"
        f"⌚ Qolgan: *{remaining:,.0f}* so'm*\n"
        f"🕒 {datetime.now().strftime('%d.%m.%Y %H:%M')}"
    )
    bot.send_photo(uid, receipt_img, caption=caption, parse_mode='Markdown',
except Exception as e:
    logging.error(f"Receipt error: {e}")
    bot.send_message(uid,
        f"✅ *To'lov kiritildi!*\n\n"
        f"💰 {amount:,.0f} so'm*\n"
        f"⌚ Qolgan: {remaining:,.0f} so'm",
        parse_mode='Markdown', reply_markup=admin_menu())

# ===== CALLBACK HANDLERS =====

@bot.callback_query_handler(func=lambda c: c.data.startswith("view:"))
def cb_view(call):
    uid = call.from_user.id
    pid = int(call.data.split(":")[1])
    db = get_db()
    prod = db.execute("SELECT * FROM products WHERE id=?", (pid,)).fetchone()
    payments = db.execute("SELECT * FROM payments WHERE product_id=? ORDER BY pai
    db.close()

    if not prod:
        bot.answer_callback_query(call.id, "Topilmadi!")
        return

    remaining = prod['total_price'] - prod['paid_amount']
    pct = min((prod['paid_amount'] / prod['total_price'] * 100) if prod['total_pr
    bar = "▀" * int(pct / 5) + "▒" * (20 - int(pct / 5))

    pay_history = ""
    for p in payments[:5]:

```

```

        icon = "👉" if p['payment_type'] == 'click' else "💰"
        dt = str(p['paid_at'])[:16].replace('T', ' ')
        pay_history += f"    {icon} {dt} - *{p['amount']:,.0f}* so'm\n"

acc_info = ""
if prod['naqd_account']:
    acc_info += f"\n💰 Naqd karta: `{prod['naqd_account']}`"
if prod['online_account']:
    bank = prod['online_bank'] or 'Online'
    acc_info += f"\n🏦 {bank}: `{prod['online_account']}`"

text = (
    f"📦 *{prod['name']}*\n"
    f"{'🏠' + prod['supplier_name'] if prod['supplier_name'] else ''}\n\n"
    f"💰 Jami nasiya: *{prod['total_price']:,.0f}* so'm*\n"
    f"✅ To'langan: *{prod['paid_amount']:,.0f}* so'm*\n"
    f"🔴 Qolgan qarz: *{remaining:,.0f}* so'm*\n\n"
    f"`{bar}` {pct:.0f}%\n"
)

if acc_info:
    text += f"\n🏠 *Hisob raqamlar:*\n{acc_info}"
if pay_history:
    text += f"\n📅 *So'nggi to'lovlar:*\n{pay_history}"

m = types.InlineKeyboardMarkup(row_width=2)
if is_admin(uid):
    m.add(
        types.InlineKeyboardButton("💸 To'lov", callback_data=f"pay:{pid}"),
        types.InlineKeyboardButton("🧾 Chek", callback_data=f"receipt:{pid}")
    )
    m.add(
        types.InlineKeyboardButton("✎ Tahrirlash", callback_data=f"edit:{pid}"),
        types.InlineKeyboardButton("🗑 O'chirish", callback_data=f"del:{pid}")
    )
    m.add(types.InlineKeyboardButton("📅 Barcha to'lovlar", callback_data=f"his"))
else:
    m.add(
        types.InlineKeyboardButton("🧾 Chek", callback_data=f"receipt:{pid}"),
        types.InlineKeyboardButton("📅 Barcha to'lovlar", callback_data=f"his")
    )

bot.answer_callback_query(call.id)
if prod['photo_file_id']:
    bot.send_photo(uid, prod['photo_file_id'], caption=text, parse_mode='Mark
else:
    bot.send_message(uid, text, parse_mode='Markdown', reply_markup=m)

```

```
# ===== TAHRIRLASH CALLBACK =====
```

```
@bot.callback_query_handler(func=lambda c: c.data.startswith("edit:"))
def cb_edit(call):
    uid = call.from_user.id
    if not is_admin(uid):
        bot.answer_callback_query(call.id, "❌ Ruksat yo'q!")
        return
    pid = int(call.data.split(":")[1])
    bot.answer_callback_query(call.id)

    m = types.InlineKeyboardMarkup(row_width=2)
    m.add(
        types.InlineKeyboardButton("📦 Nom", callback_data=f"editf:name:{pid}"),
        types.InlineKeyboardButton("👤 Yetkazuvchi", callback_data=f"editf:suppli")
    )
    m.add(
        types.InlineKeyboardButton("💰 Narx", callback_data=f"editf:price:{pid}"),
        types.InlineKeyboardButton("📅 Muddat", callback_data=f"editf:due:{pid}")
    )
    m.add(
        types.InlineKeyboardButton("💵 Naqd hisob", callback_data=f"editf:naqd:{p")
        types.InlineKeyboardButton("🏠 Online hisob", callback_data=f"editf:onlin")
    )
    m.add(
        types.InlineKeyboardButton("📷 Rasm", callback_data=f"editf:photo:{pid}"),
        types.InlineKeyboardButton("📝 Izoh", callback_data=f"editf:note:{pid}")
    )

    bot.send_message(uid, "✎ *Qaysi maydonni tahrirlaysiz?*",
                      parse_mode='Markdown', reply_markup=m)

@bot.callback_query_handler(func=lambda c: c.data.startswith("editf:"))
def cb_editf(call):
    uid = call.from_user.id
    if not is_admin(uid):
        bot.answer_callback_query(call.id, "❌ Ruksat yo'q!")
        return
    parts = call.data.split(":")
    field = parts[1]
    pid = int(parts[2])
    bot.answer_callback_query(call.id)

    data = {'product_id': pid}

    prompts = {
        'name': ("edit_name", "📦 *Yangi nom kiriting:*"),
```

```

        'supplier': ("edit_supplier", "🏢 *Yangi yetkazuvchi:*\\n_(O'tkazish mur
'price':      ("edit_price", "💰 *Yangi narx (so'mda):*",
'due':        ("edit_due", "📅 *Yangi muddat:*\\n_Format: 25.12.2024_
'naqd':       ("edit_naqd_acc", "💵 *Naqd to'lov karta/hisob:*\\n_(O'tkazi
'online':     ("edit_online_bank", "🏦 *Online bank nomi:*\\n_(Payme, Click,
'note':       ("edit_note", "📝 *Yangi izoh:*\\n_(O'tkazish mumkin)_")
'photo':      ("edit_photo", "📷 *Yangi rasm yuboring:*\\n_(O'tkazish r
    }

    if field not in prompts:
        return

    state_name, prompt = prompts[field]

    if field == 'online':
        set_state(uid, 'edit_online_bank', data)
        bot.send_message(uid, prompt, parse_mode='Markdown', reply_markup=skip_kb
    elif field == 'photo':
        set_state(uid, 'edit_photo', data)
        bot.send_message(uid, prompt, parse_mode='Markdown', reply_markup=skip_kb
    else:
        set_state(uid, state_name, data)
        kb = skip_kb() if field in ('supplier', 'due', 'naqd', 'note') else cancel
        bot.send_message(uid, prompt, parse_mode='Markdown', reply_markup=kb)

# edit_online_acc - online bank kiritilgandan keyin hisob so'raladi
@bot.message_handler(func=lambda m: get_state(m.from_user.id)['state'] == 'edit_o
def handle_edit_online_bank(msg):
    uid = msg.from_user.id
    st = get_state(uid)
    data = st['data']
    data['new_online_bank'] = msg.text.strip()
    pid = data['product_id']

    db = get_db()
    db.execute("UPDATE products SET online_bank=?, updated_at=? WHERE id=?",
               (data['new_online_bank'], datetime.now().isoformat(), pid))
    db.commit()
    db.close()

    set_state(uid, 'edit_online_acc', data)
    bot.send_message(uid, "🏦 *Online hisob raqamini kiriting:*\\n_(O'tkazish mumk
                      parse_mode='Markdown', reply_markup=skip_kb())

@bot.callback_query_handler(func=lambda c: c.data.startswith("pay:"))
def cb_pay(call):
    uid = call.from_user.id

```

```

if not is_admin(uid):
    bot.answer_callback_query(call.id, "❌ Ruksat yo'q!")
    return
pid = int(call.data.split(":")[1])
db = get_db()
prod = db.execute("SELECT name, total_price, paid_amount FROM products WHERE")
db.close()
remaining = prod['total_price'] - prod['paid_amount']
if remaining <= 0:
    bot.answer_callback_query(call.id, "✅ Bu tovar to'liq to'langan!")
    return
set_state(uid, 'pay_amount', {'product_id': pid})
bot.answer_callback_query(call.id)
bot.send_message(uid,
    f"👛 *{prod['name']}* uchun to'lov\n\n"
    f"⌚ Qolgan qarz: *{remaining:,.0f}* so'm*\n\n"
    f"💰 To'lov summasini kiriting:",
    parse_mode='Markdown', reply_markup=cancel_kb())

@bot.callback_query_handler(func=lambda c: c.data.startswith("ptype:"))
def cb_ptype(call):
    uid = call.from_user.id
    ptype = call.data.split(":")[1]
    st = get_state(uid)
    data = st['data']
    data['ptype'] = ptype
    bot.answer_callback_query(call.id)

    if ptype == 'click':
        set_state(uid, 'pay_receipt', data)
        acc_text = ""
        if data.get('online_account'):
            bank = data.get('online_bank', 'Online')
            acc_text = f"\n\n💰 *To'lov qiling:* `{data['online_account']}` ({ban
            bot.send_message(uid,
                f"📎 *Klik/Online cheki rasmini yuboring:* \n_(O'tkazish mumkin)_{acc_
                parse_mode='Markdown', reply_markup=skip_kb())
        else:
            if data.get('naqd_account'):
                bot.send_message(uid,
                    f"💵 *Naqd to'lov qiling:* \n\nKarta: `{data['naqd_account']}`",
                    parse_mode='Markdown')
                _save_payment(uid, data)

@bot.callback_query_handler(func=lambda c: c.data.startswith("receipt:"))
def cb_receipt(call):
    uid = call.from_user.id

```

```

pid = int(call.data.split(":")[1])
db = get_db()
prod = db.execute("SELECT * FROM products WHERE id=?", (pid,)).fetchone()
payments = db.execute("SELECT * FROM payments WHERE product_id=? ORDER BY pai
db.close()
remaining = prod['total_price'] - prod['paid_amount']
bot.answer_callback_query(call.id, "🧾 Chek tayyorlanmoqda...")
try:
    img = generate_receipt(dict(prod), [dict(p) for p in payments], remaining)
    caption = (
        f"🧾 *{prod['name']}* - Chek\n"
        f"🕒 Qolgan qarz: *{remaining:,.0f}* so'm*\n"
        f"🕒 {datetime.now().strftime('%d.%m.%Y %H:%M')}"
    )
    bot.send_photo(uid, img, caption=caption, parse_mode='Markdown')
except Exception as e:
    bot.send_message(uid, f"❌ Xatolik: {e}")

@bot.callback_query_handler(func=lambda c: c.data.startswith("history:"))
def cb_history(call):
    uid = call.from_user.id
    pid = int(call.data.split(":")[1])
    db = get_db()
    prod = db.execute("SELECT name FROM products WHERE id=?", (pid,)).fetchone()
    payments = db.execute("SELECT * FROM payments WHERE product_id=? ORDER BY pai
    db.close()

    if not payments:
        bot.answer_callback_query(call.id, "Hali to'lov yo'q!")
        return

    text = f"🧾 *{prod['name']}* - Barcha to'lovlar:\n\n"
    total_paid = 0
    for i, p in enumerate(payments, 1):
        icon = "👉" if p['payment_type'] == 'click' else "💰"
        dt = str(p['paid_at'][:16].replace('T', ' '))
        text += f"{i}. {icon} *{p['amount']:,.0f}* so'm* - {dt}\n"
        if p['receipt_file_id']:
            text += f"🧾 Chek mavjud\n"
        total_paid += p['amount']

    text += f"\n✅ Jami to'langan: *{total_paid:,.0f}* so'm*"

    bot.answer_callback_query(call.id)
    bot.send_message(uid, text, parse_mode='Markdown')

    for p in payments:

```

```

        if p['receipt_file_id']:
            try:
                bot.send_photo(uid, p['receipt_file_id'],
                               caption=f"📄 Chek - {str(p['paid_at'][:10])} | {p[
            except:
                pass

@bot.callback_query_handler(func=lambda c: c.data.startswith("del:"))
def cb_delete(call):
    uid = call.from_user.id
    if not is_admin(uid):
        bot.answer_callback_query(call.id, "❌ Ruksat yo'q!")
        return
    pid = int(call.data.split(":")[1])
    m = types.InlineKeyboardMarkup()
    m.add(
        types.InlineKeyboardButton("✅ Ha, o'chirish", callback_data=f"delok:{pid}"),
        types.InlineKeyboardButton("❌ Yo'q", callback_data="delno")
    )
    bot.answer_callback_query(call.id)
    bot.send_message(uid, "⚠️ *Haqiqatan ham o'chirmoqchimisiz?*\nBarcha to'lov t
                      parse_mode='Markdown', reply_markup=m)

@bot.callback_query_handler(func=lambda c: c.data.startswith("delok:"))
def cb_delok(call):
    uid = call.from_user.id
    pid = int(call.data.split(":")[1])
    db = get_db()
    db.execute("DELETE FROM payments WHERE product_id=?", (pid,))
    db.execute("DELETE FROM reminders WHERE product_id=?", (pid,))
    db.execute("DELETE FROM products WHERE id=?", (pid,))
    db.commit()
    db.close()
    bot.answer_callback_query(call.id, "🗑️ O'chirildi!")
    bot.send_message(uid, "🗑️ Tovar o'chirildi.", reply_markup=admin_menu())

@bot.callback_query_handler(func=lambda c: c.data == "delno")
def cb_delno(call):
    bot.answer_callback_query(call.id, "❌ Bekor qilindi")

@bot.callback_query_handler(func=lambda c: c.data == "add_worker")
def cb_add_worker(call):
    uid = call.from_user.id
    if not is_admin(uid):
        bot.answer_callback_query(call.id, "❌ Ruksat yo'q!")
        return
    set_state(uid, 'add_worker_id')

```

```

bot.answer_callback_query(call.id)
bot.send_message(uid,
    "👤 *Xodim qo'shish*\n\n"
    "Xodimingizga botga `/start` yuborishni so'rang.\n"
    "Keyin uning *Telegram ID* sini yuboring:\n\n"
    "*ID ni bilish uchun @userinfobot ga /start yuboring*",
    parse_mode='Markdown', reply_markup=cancel_kb())

@bot.callback_query_handler(func=lambda c: c.data == "remove_worker")
def cb_remove_worker(call):
    uid = call.from_user.id
    if not is_admin(uid):
        bot.answer_callback_query(call.id, "Ruxsat yo'q!")
        return
    db = get_db()
    workers = db.execute("SELECT user_id, full_name FROM users WHERE role='worker'")
    db.close()
    if not workers:
        bot.answer_callback_query(call.id, "Xodim yo'q!")
        return
    m = types.InlineKeyboardMarkup()
    for w in workers:
        m.add(types.InlineKeyboardButton(f"👤 {w['full_name']}", callback_data=f"{w['full_name']}"))
    bot.answer_callback_query(call.id)
    bot.send_message(uid, "👤 Qaysi xodimni o'chirasiz?", reply_markup=m)

@bot.callback_query_handler(func=lambda c: c.data.startswith("delworker:"))
def cb_delworker(call):
    uid = call.from_user.id
    wid = int(call.data.split(":")[1])
    db = get_db()
    db.execute("DELETE FROM users WHERE user_id=? AND role='worker'", (wid,))
    db.commit()
    db.close()
    bot.answer_callback_query(call.id, "✅ O'chirildi!")
    bot.send_message(uid, "✅ Xodim o'chirildi.", reply_markup=admin_menu())

# ===== WEB DASHBOARD =====

WEB_HTML = '''<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>☕ Kafe Nasiya Daftari</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Syne:wght@400;600;700;800&fa
'''

```

```

<style>
  :root {
    --bg: #080c12;
    --surface: #0f1520;
    --card: #141c28;
    --border: #1e2d42;
    --accent: #f0883e;
    --accent2: #58a6ff;
    --green: #3fb950;
    --red: #f85149;
    --text: #e6edf3;
    --muted: #7d8fa8;
    --radius: 16px;
  }
  *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    background: var(--bg);
    color: var(--text);
    font-family: 'Syne', sans-serif;
    min-height: 100vh;
    background-image:
      radial-gradient(ellipse at 20% 10%, rgba(240,136,62,0.08) 0%, transparent 5
      radial-gradient(ellipse at 80% 80%, rgba(88,166,255,0.06) 0%, transparent 5
  }

  /* LOGIN */
  .login-wrap {
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
  }
  .login-box {
    background: var(--card);
    border: 1px solid var(--border);
    border-radius: 24px;
    padding: 48px 40px;
    width: 380px;
    text-align: center;
  }
  .login-box .logo { font-size: 48px; margin-bottom: 16px; }
  .login-box h1 { font-size: 22px; font-weight: 800; margin-bottom: 6px; }
  .login-box p { color: var(--muted); font-size: 14px; margin-bottom: 32px; }
  .login-box input {
    width: 100%; padding: 14px 18px;
    background: var(--surface); border: 1px solid var(--border);
  }

```

```

border-radius: 12px; color: var(--text);
font-family: 'DM Mono', monospace; font-size: 15px;
margin-bottom: 16px; outline: none;
transition: border-color .2s;
}
.login-box input:focus { border-color: var(--accent); }
.btn {
width: 100%; padding: 14px;
background: linear-gradient(135deg, var(--accent), #e07020);
border: none; border-radius: 12px;
color: #fff; font-family: 'Syne', sans-serif;
font-weight: 700; font-size: 15px; cursor: pointer;
transition: opacity .2s, transform .1s;
}
.btn:hover { opacity: .9; }
.btn:active { transform: scale(.98); }
.login-err { color: var(--red); font-size: 13px; margin-top: 12px; }

/* DASHBOARD */
.dash { display: none; }
.topbar {
display: flex; align-items: center; justify-content: space-between;
padding: 20px 32px;
border-bottom: 1px solid var(--border);
background: rgba(15,21,32,.8);
backdrop-filter: blur(12px);
position: sticky; top: 0; z-index: 100;
}
.topbar .brand { font-size: 18px; font-weight: 800; }
.topbar .brand span { color: var(--accent); }
.topbar .logout {
background: transparent; border: 1px solid var(--border);
color: var(--muted); padding: 8px 16px; border-radius: 8px;
font-family: 'Syne', sans-serif; font-size: 13px; cursor: pointer;
transition: all .2s;
}
.topbar .logout:hover { border-color: var(--red); color: var(--red); }

.container { max-width: 1200px; margin: 0 auto; padding: 32px 24px; }

/* STATS */
.stats-grid {
display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
gap: 16px; margin-bottom: 32px;
}
.stat-card {
background: var(--card); border: 1px solid var(--border);

```

```

border-radius: var(--radius); padding: 24px;
position: relative; overflow: hidden;
transition: transform .2s, border-color .2s;
}
.stat-card:hover { transform: translateY(-3px); border-color: #2a3f58; }
.stat-card::before {
  content: ''; position: absolute;
  top: 0; left: 0; right: 0; height: 3px;
}
.stat-card.orange::before { background: var(--accent); }
.stat-card.blue::before { background: var(--accent2); }
.stat-card.green::before { background: var(--green); }
.stat-card.red::before { background: var(--red); }

.stat-label { font-size: 12px; color: var(--muted); font-weight: 600; letter-sp
.stat-val { font-size: 28px; font-weight: 800; line-height: 1; }
.stat-val.orange { color: var(--accent); }
.stat-val.blue { color: var(--accent2); }
.stat-val.green { color: var(--green); }
.stat-val.red { color: var(--red); }

/* PROGRESS */
.progress-wrap { background: var(--card); border: 1px solid var(--border); bord
.progress-header { display: flex; justify-content: space-between; align-items:
.progress-title { font-size: 16px; font-weight: 700; }
.progress-pct { font-family: 'DM Mono', monospace; color: var(--accent); font-s
.progress-bar { height: 12px; background: var(--surface); border-radius: 99px;
.progress-fill { height: 100%; border-radius: 99px; background: linear-gradient

/* FILTERS */
.filters { display: flex; gap: 10px; margin-bottom: 24px; flex-wrap: wrap; alig
.filter-btn {
  background: var(--card); border: 1px solid var(--border);
  color: var(--muted); padding: 8px 16px; border-radius: 99px;
  font-family: 'Syne', sans-serif; font-size: 13px; cursor: pointer;
  transition: all .2s;
}
.filter-btn.active, .filter-btn:hover { border-color: var(--accent); color: var
.search-box {
  flex: 1; min-width: 200px;
  background: var(--card); border: 1px solid var(--border);
  border-radius: 99px; padding: 8px 18px;
  color: var(--text); font-family: 'Syne', sans-serif; font-size: 14px; outline
  transition: border-color .2s;
}
.search-box:focus { border-color: var(--accent2); }

```

```

/* TABLE */
.table-wrap {
  background: var(--card); border: 1px solid var(--border);
  border-radius: var(--radius); overflow: hidden;
}
table { width: 100%; border-collapse: collapse; }
thead { background: var(--surface); }
th { padding: 14px 18px; font-size: 11px; font-weight: 700; text-transform: uppercase; }
td { padding: 16px 18px; font-size: 14px; border-top: 1px solid var(--border); }
tr:hover td { background: rgba(255,255,255,.02); }

.badge {
  display: inline-block; padding: 4px 10px; border-radius: 99px;
  font-size: 11px; font-weight: 700;
}
.badge.paid { background: rgba(63,185,80,.15); color: var(--green); border: 1px solid var(--border); }
.badge.unpaid { background: rgba(248,81,73,.15); color: var(--red); border: 1px solid var(--border); }
.badge.partial { background: rgba(240,136,62,.15); color: var(--accent); border: 1px solid var(--border); }

.mini-bar { height: 6px; background: var(--surface); border-radius: 99px; min-width: 100%; }
.mini-fill { height: 100%; border-radius: 99px; }

.acc-info { font-family: 'DM Mono', monospace; font-size: 11px; color: var(--muted); }
.acc-info span { color: var(--accent2); }

.expand-btn {
  background: transparent; border: 1px solid var(--border);
  color: var(--muted); padding: 5px 12px; border-radius: 8px;
  font-size: 12px; cursor: pointer; transition: all .2s;
}
.expand-btn:hover { border-color: var(--accent2); color: var(--accent2); }

/* DETAIL PANEL */
.detail-panel {
  display: none; background: var(--surface);
  border-top: 1px solid var(--border);
}
.detail-panel.open { display: table-row; }
.detail-inner { padding: 20px 24px; }
.detail-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 16px; }
.detail-section h4 { font-size: 11px; text-transform: uppercase; letter-spacing: .02em; }
.pay-item {
  display: flex; justify-content: space-between; align-items: center;
  background: var(--card); border-radius: 10px; padding: 10px 14px;
  margin-bottom: 8px; font-size: 13px;
}
.pay-item .pay-type { color: var(--muted); font-size: 11px; }

```

```

.pay-item .pay-amount { font-weight: 700; color: var(--green); font-family: 'DM
.acc-box {
    background: var(--card); border-radius: 10px; padding: 12px 14px; margin-bott
    font-family: 'DM Mono', monospace; font-size: 12px;
}
.acc-box .acc-label { color: var(--muted); font-size: 10px; text-transform: upp
.acc-box .acc-num { color: var(--text); font-size: 14px; }

.no-data { text-align: center; padding: 60px; color: var(--muted); font-size: 1

/* LOADER */
.loader { text-align: center; padding: 60px; color: var(--muted); }
.spin { display: inline-block; width: 32px; height: 32px; border: 3px solid var
@keyframes spin { to { transform: rotate(360deg); } }

@media (max-width: 768px) {
    .topbar { padding: 14px 16px; }
    .container { padding: 16px; }
    .stats-grid { grid-template-columns: 1fr 1fr; }
    .detail-grid { grid-template-columns: 1fr; }
    table { font-size: 12px; }
    th, td { padding: 10px 10px; }
}
</style>
</head>
<body>

<!-- LOGIN -->
<div class="login-wrap" id="loginWrap">
    <div class="login-box">
        <div class="logo"><img alt="Coffee cup icon" data-bbox="338 625 358 638"/></div>
        <h1>Kafe Nasiya Daftari</h1>
        <p>Web Dashboard – Admin kirish</p>
        <input type="password" id="passInput" placeholder="Parolni kiriting..." onkey
        <button class="btn" onclick="login()">Kirish →</button>
        <div class="login-err" id="loginErr"></div>
    </div>
</div>

<!-- DASHBOARD -->
<div class="dash" id="dash">
    <div class="topbar">
        <div class="brand"><img alt="Coffee cup icon" data-bbox="345 852 365 865"/> <span>Nasiya</span> Daftari</div>
        <button class="logout" onclick="logout()">Chiqish</button>
    </div>
    <div class="container">

```

```

<!-- STATS -->
<div class="stats-grid" id="statsGrid">
    <div class="stat-card orange"><div class="stat-label">Jami tovarlar</div><d
    <div class="stat-card red"><div class="stat-label">Jami nasiya</div><div cl
    <div class="stat-card green"><div class="stat-label">To'langan</div><div cl
    <div class="stat-card blue"><div class="stat-label">Qolgan qarz</div><div c
</div>

<!-- PROGRESS -->
<div class="progress-wrap">
    <div class="progress-header">
        <div class="progress-title">Umumiy to'lov holati</div>
        <div class="progress-pct" id="pPct">0%</div>
    </div>
    <div class="progress-bar"><div class="progress-fill" id="pFill" style="width: 0%;></div>

<!-- FILTERS -->
<div class="filters">
    <button class="filter-btn active" onclick="setFilter('all',this)">Hammasi</button>
    <button class="filter-btn" onclick="setFilter('unpaid',this)">Qarzli</button>
    <button class="filter-btn" onclick="setFilter('paid',this)">To'langan</button>
    <input class="search-box" type="text" placeholder="🔍 Qidirish..." id="search">
</div>

<!-- TABLE -->
<div class="table-wrap">
    <div class="loader" id="loader"><div class="spin"></div><p style="margin-top: 20px;">
    <table id="prodTable" style="display:none">
        <thead>
            <tr>
                <th>#</th>
                <th>Tovar nomi</th>
                <th>Jami</th>
                <th>To'langan</th>
                <th>Qolgan</th>
                <th>Holat</th>
                <th>Muddat</th>
                <th></th>
            </tr>
        </thead>
        <tbody id="prodBody"></tbody>
    </table>
    <div class="no-data" id="noData" style="display:none">🚫 Tovar topilmadi</div>
</div>
</div>

```

</div>

<script>

```
let token = sessionStorage.getItem('wt') || '';
```

```
let products = [];
```

```
let filter = 'all';
```

```
function fmt(n) {
```

```
    return Number(n).toLocaleString('uz') + ' so\'m';
```

```
}
```

```
function fmtShort(n) {
```

```
    if (n >= 1e9) return (n/1e9).toFixed(1) + ' mlrd';
```

```
    if (n >= 1e6) return (n/1e6).toFixed(1) + ' mln';
```

```
    return Number(n).toLocaleString('uz');
```

```
}
```

```
async function login() {
```

```
    const p = document.getElementById('passInput').value;
```

```
    const r = await fetch('/api/login', {method:'POST', headers: {'Content-Type': 'ap
```

```
    const d = await r.json();
```

```
    if (d.ok) {
```

```
        token = d.token;
```

```
        sessionStorage.setItem('wt', token);
```

```
        showDash();
```

```
    } else {
```

```
        document.getElementById('loginErr').textContent = '❌ Parol noto\'g\'ri!';
```

```
    }
```

```
}
```

```
function logout() {
```

```
    token = ''; sessionStorage.removeItem('wt');
```

```
    document.getElementById('loginWrap').style.display = 'flex';
```

```
    document.getElementById('dash').style.display = 'none';
```

```
}
```

```
async function showDash() {
```

```
    document.getElementById('loginWrap').style.display = 'none';
```

```
    document.getElementById('dash').style.display = 'block';
```

```
    await loadData();
```

```
}
```

```
async function loadData() {
```

```
    const r = await fetch('/api/products', {headers: {'X-Token': token}});
```

```
    if (r.status === 401) { logout(); return; }
```

```
    const d = await r.json();
```

```
    products = d.products || [];
```

```
    renderStats();
```

```

renderTable();
document.getElementById('loader').style.display = 'none';
document.getElementById('prodTable').style.display = 'table';
}

function renderStats() {
  let count = products.length;
  let total = products.reduce((s,p) => s+p.total_price, 0);
  let paid = products.reduce((s,p) => s+p.paid_amount, 0);
  let rem = total - paid;
  let pct = total > 0 ? Math.min(paid/total*100, 100) : 0;

  document.getElementById('sCount').textContent = count + ' ta';
  document.getElementById('sTotal').textContent = fmtShort(total);
  document.getElementById('sPaid').textContent = fmtShort(paid);
  document.getElementById('sRem').textContent = fmtShort(rem);
  document.getElementById('pPct').textContent = pct.toFixed(1) + '%';
  setTimeout(() => document.getElementById('pFill').style.width = pct + '%', 100)
}

function setFilter(f, btn) {
  filter = f;
  document.querySelectorAll('.filter-btn').forEach(b => b.classList.remove('active'));
  btn.classList.add('active');
  renderTable();
}

function renderTable() {
  const q = document.getElementById('searchBox').value.toLowerCase();
  let rows = products.filter(p => {
    const rem = p.total_price - p.paid_amount;
    if (filter === 'paid' && rem > 0) return false;
    if (filter === 'unpaid' && rem <= 0) return false;
    if (q && !p.name.toLowerCase().includes(q) && !(p.supplier_name||'').toLowerCase().includes(q)) return false;
    return true;
  });

  const tbody = document.getElementById('prodBody');
  if (rows.length === 0) {
    document.getElementById('prodTable').style.display = 'none';
    document.getElementById('noData').style.display = 'block';
    return;
  }
  document.getElementById('prodTable').style.display = 'table';
  document.getElementById('noData').style.display = 'none';

  tbody.innerHTML = rows.map((p, i) => {

```

```

const rem = p.total_price - p.paid_amount;
const pct = p.total_price > 0 ? Math.min(p.paid_amount/p.total_price*100, 100
const barColor = pct >= 90 ? '#3fb950' : pct >= 50 ? '#f0883e' : '#f85149';
let badge = rem <= 0
  ? '<span class="badge paid">✅ To\'liq</span>'
  : pct > 0
    ? '<span class="badge partial">⚠️ Qisman</span>'
    : '<span class="badge unpaid">🚫 To\'lanmagan</span>';
const due = p.due_date ? p.due_date.slice(0,10) : '-';

let accInfo = '';
if (p.naqd_account) accInfo += `<div class="acc-info">💰 Naqd: <span>${p.naqd
if (p.online_account) {
  const bank = p.online_bank || 'Online';
  accInfo += `<div class="acc-info">🏦 ${bank}: <span>${p.online_account}</sp
}

return `
<tr>
  <td style="color:var(--muted);font-family:'DM Mono',monospace">${i+1}</td>
  <td>
    <div style="font-weight:700">${p.name}</div>
    ${p.supplier_name ? `<div style="color:var(--muted);font-size:12px">🏢 ${
      ${accInfo}
    }
  </td>
  <td style="font-family:'DM Mono',monospace">${fmt(p.total_price)}</td>
  <td style="font-family:'DM Mono',monospace;color:var(--green)">${fmt(p.paid
  <td style="font-family:'DM Mono',monospace;color:${rem>0?'var(--red)':'var(
  <td>
    ${badge}
    <div class="mini-bar" style="margin-top:6px">
      <div class="mini-fill" style="width:${pct}%;background:${barColor}"></d
    </div>
  </td>
  <td style="color:var(--muted);font-size:13px">${due}</td>
  <td><button class="expand-btn" onclick="toggleDetail(${p.id})">Ko'rish</but
</tr>
<tr id="detail-${p.id}" class="detail-panel">
  <td colspan="8">
    <div class="detail-inner" id="detail-inner-${p.id}">
      <div style="color:var(--muted);font-size:13px">Yuklanmoqda...</div>
    </div>
  </td>
</tr>`;
}).join('');
}

```

```

async function toggleDetail(id) {
  const panel = document.getElementById('detail-' + id);
  const inner = document.getElementById('detail-inner-' + id);

  if (panel.classList.contains('open')) {
    panel.classList.remove('open');
    return;
  }
  panel.classList.add('open');

  const r = await fetch('/api/product/' + id, {headers: {'X-Token': token}});
  const d = await r.json();
  const p = d.product;
  const pays = d.payments || [];

  let payHtml = pays.length === 0
    ? '<div style="color:var(--muted);font-size:13px">Hali to\'lov yo\'q</div>'
    : pays.map(pay => `
      <div class="pay-item">
        <div>
          <div>${pay.payment_type==='click'?'🏠 Online':'$🏠 Naqd'}</div>
          <div class="pay-type">${pay.paid_at.slice(0,16).replace('T',' ')}</div>
        </div>
        <div class="pay-amount">+${fmt(pay.amount)}</div>
      </div>`).join('');

  let accHtml = '';
  if (p.naqd_account) accHtml += `
    <div class="acc-box">
      <div class="acc-label">🏠 Naqd to\'lov kartasi</div>
      <div class="acc-num">${p.naqd_account}</div>
    </div>`;
  if (p.online_account) accHtml += `
    <div class="acc-box">
      <div class="acc-label">🏠 ${p.online_bank||'Online'}</div>
      <div class="acc-num">${p.online_account}</div>
    </div>`;
  if (!accHtml) accHtml = '<div style="color:var(--muted);font-size:13px">Hisob r

  let noteHtml = p.note ? `<div style="color:var(--muted);font-size:13px;margin-t

  inner.innerHTML = `
    <div class="detail-grid">
      <div class="detail-section">
        <h4>📅 To\'lovlar tarixi</h4>
        ${payHtml}
      </div>

```

```

        <div class="detail-section">
            <h4>💰 Hisob raqamlar</h4>
            ${accHtml}
            ${noteHtml}
        </div>
    </div>`;
}

```

```

// Auto-login if token exists
if (token) showDash();
</script>
</body>
</html>'''

```

```
# ===== API ROUTES =====
```

```
import hashlib, secrets
```

```
active_tokens = {}
```

```

@app.route('/')
def web_index():
    return render_template_string(WEB_HTML)

```

```

@app.route('/api/login', methods=['POST'])
def api_login():
    data = request.get_json()
    if data and data.get('password') == WEB_SECRET:
        tok = secrets.token_hex(24)
        active_tokens[tok] = datetime.now()
        return jsonify({'ok': True, 'token': tok})
    return jsonify({'ok': False}), 401

```

```

def check_token():
    tok = request.headers.get('X-Token', '')
    return tok in active_tokens

```

```

@app.route('/api/products')
def api_products():
    if not check_token():
        return jsonify({'error': 'Unauthorized'}), 401
    db = get_db()
    rows = db.execute(
        "SELECT id, name, supplier_name, total_price, paid_amount, due_date, "
        "naqd_account, online_account, online_bank, note FROM products ORDER BY c"
    ).fetchall()
    db.close()

```

```

        return jsonify({'products': [dict(r) for r in rows]})

@app.route('/api/product/<int:pid>')
def api_product(pid):
    if not check_token():
        return jsonify({'error': 'Unauthorized'}), 401
    db = get_db()
    prod = db.execute("SELECT * FROM products WHERE id=?", (pid,)).fetchone()
    pays = db.execute(
        "SELECT amount, payment_type, paid_at FROM payments WHERE product_id=? OR
        (pid,)"
    ).fetchall()
    db.close()
    if not prod:
        return jsonify({'error': 'Not found'}), 404
    return jsonify({'product': dict(prod), 'payments': [dict(p) for p in pays]})

def run_web():
    app.run(host='0.0.0.0', port=WEB_PORT, debug=False, use_reloader=False)

# ===== ISHGA TUSHIRISH =====

if __name__ == '__main__':
    init_db()
    print("☕ Kafe Nasiya Daftari Bot ishga tushdi!")
    print(f"🌐 Web dashboard: http://0.0.0.0:{WEB_PORT}")
    print(f"🔑 Web parol: {WEB_SECRET}")

    # Eslatma thread
    t1 = threading.Thread(target=reminder_loop, daemon=True)
    t1.start()

    # Web server thread
    t2 = threading.Thread(target=run_web, daemon=True)
    t2.start()

    bot.infinity_polling(timeout=30, long_polling_timeout=20)

```